1. Introduction: Looking Under the Hood

Most of the time, we just click on folders in Windows, and the computer does all the work for us. But those flashy graphics actually hide how the computer truly operates. For my project, I wanted to see what happens behind the scenes.

So, I built the **Simple File Explorer**. It is a text-based (CLI) program written in C++ that lets you browse folders, make files, delete things, and check file details right from the terminal.

Why use this? * It's incredibly fast: Because there are no graphics or icons to load, it runs instantly.

- **It's educational:** It doesn't rely on hidden Windows shortcuts. Every time you type a command, the program talks directly to the Operating System to get the job done.

---

2. What I Learned and Built

Building this was a lot harder than just writing a simple script. I had to figure out how to make my code talk to the actual computer hardware. Here are the main skills I developed while making this project work:

Getting C++ to Talk to the OS

To make this work, I used the modern C++17 `<filesystem>` library. This taught me how an Operating System actually thinks. Instead of just "making a folder," I learned how my program sends a request to the Windows Kernel to find empty space on the hard drive and allocate it for a new directory. I also learned how to pull file details (like size and type) straight from the disk.

Fixing Broken Tools

One of my biggest flexes in this project wasn't even writing the code—it was fixing my setup. The default compiler in my IDE (Code::Blocks) was too old to understand modern C++.

- I took the initiative to completely replace it with a new MSYS2 compiler.
- When the IDE got confused and mashed the old and new file paths together (causing the whole thing to break), I went into the toolchain settings and manually re-linked everything so the compiler could run. This proved I can troubleshoot my actual work environment, not just my code.

Keeping Code Organized

When I started, I shoved all my code into one giant `main.cpp` file. When I tried to run it, the compiler threw linking errors because the file was too messy. I learned my lesson and split

the project into three clean pieces: a header file (`FileManager.h`) for my rules, a `FileManager.cpp` file for the actual work, and a simple `main.cpp` for the user menu. This made my code look professional and easy to read.

Stopping Crashes

While testing, I realized that if I tried to scan a locked system folder (like `System32`), the OS would block me, and my whole program would crash. I fixed this by adding error-handling loops. I told the code, "If Windows says you don't have permission to read a file, just skip it and keep going." This made the app much more stable and proved I could handle unexpected bugs.

---

3. Conclusion

Building the Simple File Explorer was a huge learning experience. It forced me to deal with annoying compiler bugs, organize my code properly, and figure out how programs actually communicate with Windows. I am walking away from this project with a much better understanding of how computers work from the inside out.